

Homework Assignment B

This document contains the write-ups for the three challenges of **Assignment B**.

Problem ID	Lab Name	Captured Flag	Steps
Problem 1	Liars Dice	<code>fsuCTF{800t57r4p_8i1l_y0u_4r3_4_li4r_4nd_y0u_will_5p3nd_4n_37ernity_0n_7hi5_5hip}</code>	- Connect to the server and record the connection timestamp - Read the dice shown in round 1 - Brute-force nearby seeds until finding the one that reproduces those dice - Use the seed to predict all 100 rounds in advance - For each round, bid the maximum true statement so the AI is forced to call liar and lose
Problem 2	RSA Master	<code>fsuCTF{You_Can_Do_CRT_For_Any_E_If_You_Have_Enough_Ciphertexts}</code>	- Connect to the server 7 times and collect 7 pairs (ciphertext, n), noting that e = 7 is always the same - Apply CRT to the 7 pairs to recover m^7 as a plain integer - Compute the exact 7th root of the result - Decode the integer from hex to text to get the flag
Problem 3	Jack The Ripper	<code>fsuCTF{John_Work_Well_With_Rockyou}</code>	- Use zip2john to extract the hash from the zip - Run John the Ripper with the rockyou.txt wordlist against the hash - Unzip the file using the cracked password and read flag.txt

PROBLEM 1 - Liars Dice

In this challenge, we are given a Python script and a server to connect to. The goal is to win 100 rounds of Liar's Dice against an AI.

1. Analysis

The first thing I did was connect to the server to see what we were dealing with:

```
nc ctf.cs.fsu.edu 65053
```

```
Welcome to Liar's Dice!
Rules: Bid on the total number of dice matching a certain face value.
Bids must increase in quantity, or same quantity with a higher face value.
Format your bids as 'Quantity Face' (e.g., '3 4' for three 4s).
Type 'liar' to challenge the AI's bid.
```

```
--- ROUND 1 / 100 ---
Score - You: 0 | AI: 0
```

```
Your dice: [2, 3, 1, 5, 4, 4, 6, 2, 1, 3]
Enter your bid (e.g., '2 5') or 'liar':
```

100 rounds is way too many to play manually, so I read the Python file carefully to understand how the game works.

Looking at the AI logic, it looks at all the dice on the table and only makes bids it knows are true:

```
all_dice = player_dice + ai_dice
...
if all_dice.count(test_v) >= test_q:
    safe_bid = (test_q, test_v)
```

Apparently the AI is seeing our dice. This means the AI has a weakness: if we make a bid that is the maximum possible true statement, the AI cannot find any higher safe bid and is forced to call "liar" and lose, because our bid was true.

The objective will then be to know all the dice are before we play. The information we need to do it is in this line:

```
random.seed(int(time.time()*1000000))
```

The seed for the random number generator is just the current time in microseconds at the moment the server starts the game. If we know that timestamp, we can reproduce every single dice roll of the entire game before playing a single round, because Python's `random` module always produces the same output from the same input.

We can't know the exact microsecond, but we can estimate it from our side: we record the time just before and after connecting, and use the average. Then we brute-force nearby values until we find the seed that produces the same dice we see in round 1.

2. Decipher

I wrote a script that does all of this automatically:

```
import socket, time, random, re

HOST = 'ctf.cs.fsu.edu'
PORT = 65053

s = socket.socket()
s.settimeout(10)
t1 = time.time()
s.connect((HOST, PORT))
t2 = time.time()
my_time = int((t1 + t2) / 2 * 1000000)
print("Connected, timestamp:", my_time)

data = b''
try:
    while b'Enter your bid' not in data:
        data += s.recv(4096)
except socket.timeout:
    pass
data = data.decode()
print(data)
```

```

my_dice = list(map(int, re.search(r'Your dice: \[[^\]]+\]', data).group(1).split(',')))
print("My dice:", my_dice)

print("Searching seed...")
seed = None
for delta in range(-5000000, 5000000):
    rng = random.Random(my_time + delta)
    if [rng.randint(1,6) for _ in range(10)] == my_dice:
        seed = my_time + delta
        print("Seed found:", seed)
        break

rng = random.Random(seed)
rounds = []
for _ in range(100):
    p = [rng.randint(1,6) for _ in range(10)]
    a = [rng.randint(1,6) for _ in range(10)]
    rounds.append(p + a)

for i, all_dice in enumerate(rounds):
    counts = {v: all_dice.count(v) for v in range(1, 7)}
    best_q = max(counts.values())
    best_v = max(v for v in range(1, 7) if counts[v] == best_q)
    print(f"Round {i+1}: bidding {best_q} {best_v}")
    s.sendall(f"{best_q} {best_v}\n".encode())

data = b''
marker = b'Enter your bid' if i < 99 else b''
try:
    while marker not in data:
        data += s.recv(4096)
except socket.timeout:
    pass
print(data.decode())

s.close()

```

Running it:

```

--- ROUND 99 / 100 ---
Score - You: 98 | AI: 0
Your dice: [1, 6, 3, 5, 6, 3, 2, 1, 2, 5]

Round 99: bidding 6 3

AI: I call Liar!
Revealing all dice: [1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 3, 3, 4, 4, 4, 5, 5, 6, 6]
There are 6x 3's.
You win the round!

--- ROUND 100 / 100 ---
Score - You: 99 | AI: 0
Your dice: [4, 1, 6, 6, 6, 6, 3, 5, 5, 4]

Round 100: bidding 6 6

AI: I call Liar!
Revealing all dice: [1, 1, 2, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 6, 6, 6, 6, 6, 6]
There are 6x 6's.
You win the round!

=====
🎉 CONGRATULATIONS! 🎉
You have successfully survived 100 rounds of Liar's Dice!
=====
fsuCTF{800t57r4p_8i1l_y0u_4r3_4_1i4r_4nd_y0u_will_5p3nd_4n_37ernity_0n_7hi5_5hip}

```

The script found the seed (`1776093203548029`), predicted all dice rolls in advance, and won every single remaining round 99-0.

Flag: `fsuCTF{800t57r4p_8i1l_y0u_4r3_4_1i4r_4nd_y0u_will_5p3nd_4n_37ernity_0n_7hi5_5hip}`

PROBLEM 2 - RSA Master

In this challenge, we are given a remote server to connect to and nothing else.

1. Analysis

The first thing I did was connect to the server to see what it gives me:

```
nc ctf.cs.fsu.edu 65054
```

Output:

```

ciphertext =
10185350424199306250949295678489836948288458873570334214363888222901857199326489603186292807094
54197938124722720698245416454319653817263789415347456181064
public_key = (7,
75139066975739316154030223468167447448262662114958324597127075691929266438112273539603197947596
75978499748659019265975786551942099969945424080734362930937)

```

So the public key is (e, n) where $e = 7$.

I noticed that every time I reconnect to the server, it gives me a different ciphertext and a different n , but e is always 7. That means the server is probably encrypting the same message m each time, just with a different n .

If the same message m is encrypted with the same e but with e different modulus, I can use the **Chinese Remainder Theorem (CRT)** to combine the results and recover m^e without any modulus. Then I just compute the exact e -th root to get m .

For this to work I need exactly 7 pairs (ciphertext, n) . I connected 7 times to the server and collected them all:

```
nc ctf.cs.fsu.edu 65054
```

#	ciphertext
1	1018535042419930625094929567848983694828845887357033421436388822290185719932648960318629280709454197938124722720698245416454319653817263789415347456181064,
2	3133118911581818596560345466066239553730274995399945403331375793698913417200357927619237194363574661436910797998002797918496993358475764293425409533047598,
3	3601054754196131012287556495152900493673987913809337313955837392413149190146640470531090973024225456516746781900064259028312514893612842514425698094573314,
4	2898473413093990888561540167227317518848963628651407444115580237830653872011511660224598245187471328929116054690952868154414738941010180275864104316264906,
5	8708235974053954942279770350448180107214788534330896911145359033116252765131387741368181163105233467638633753642345533364815204068975118910675637247979316,
6	921814350879819305323536363687681301301454357850192340722668809676455161586373474763191261786818262078104179691039756613058850394297322362591911825623785,
7	2961459301489772520014958213087277413844536937862996322703529281373878184119327124941928083995904195402523118586320115369595680712107134682257743781353663,

2. Decipher

With the 7 pairs collected, I applied the attack. The script does three things:

- Apply CRT to the 7 pairs (ci, ni) to get m⁷ as a plain integer
- Compute the exact 7th root of that integer
- Decode the result from hex to text

```
from sympy.ntheory.modular import crt
import gmpy2
```

```
cs = [
```

```
1018535042419930625094929567848983694828845887357033421436388822290185719932648960318629280709454197938124722720698245416454319653817263789415347456181064,
```

```
3133118911581818596560345466066239553730274995399945403331375793698913417200357927619237194363574661436910797998002797918496993358475764293425409533047598,
```

```
3601054754196131012287556495152900493673987913809337313955837392413149190146640470531090973024225456516746781900064259028312514893612842514425698094573314,
```

```
2898473413093990888561540167227317518848963628651407444115580237830653872011511660224598245187471328929116054690952868154414738941010180275864104316264906,
```

```
8708235974053954942279770350448180107214788534330896911145359033116252765131387741368181163105233467638633753642345533364815204068975118910675637247979316,
```

```
921814350879819305323536363687681301301454357850192340722668809676455161586373474763191261786818262078104179691039756613058850394297322362591911825623785,
```

```
2961459301489772520014958213087277413844536937862996322703529281373878184119327124941928083995904195402523118586320115369595680712107134682257743781353663,
```

```
]
```

```
ns = [
```

```
75139066975739316154030223468167447448262662114958324597127075691929266438112273539603197947596
```

```

75978499748659019265975786551942099969945424080734362930937,

68960354959436013156719971063457941481607731205759899254723494079957426339408941228283889652988
31841290930536799792638742283236537974942927522544910242363,

69970538636509771203830346031555289805531723001990249988224140913019814284737461946639364466309
39673945351344499567235380671904996712853409285829581695037,

76732218018306392447913030036511736948526708406165080736422456867493002285821009781982660093519
12525236341539588053272735240993415824953401421567064687621,

10219965279349923585250532321241621964017079317150253326618979321632481450238323613027066199086
752346724627509161043362829639871937903956960961539681900071,

39092164575519769634856519938938862094751479408527476144814472634439301870177740190627998533071
60087182495384261887511442279195761425101035458360520179681,

98018812374475023114727727062489184584185641497791863071798248752483313258469104163279877284201
82441544025862273171696474656420566771127417661702903210939,
]

result, mod = crt(ns, cs)
print('CRT ok')

m, exact = gmpy2.iroot(result, 7)
print('Exact root:', exact)
print('m (hex):', hex(m))
print('Flag:', bytes.fromhex(hex(m)[2:]))

```

Output:

```

CRT ok
Exact root: True
m (hex):
0x6673754354467b596f755f43616e5f446f5f4352545f466f725f416e795f455f49665f596f755f486176655f456e6
f7567685f43697068657274657874737d
Flag: b'fsuCTF{You_Can_Do_CRT_For_Any_E_If_You_Have_Enough_Ciphertexts}'

```

`exact = True` confirms that `m7` was recovered without any modulo reduction and the root is exact.

Flag: `fsuCTF{You_Can_Do_CRT_For_Any_E_If_You_Have_Enough_Ciphertexts}`

PROBLEM 3 - Jack the Ripper

In this challenge, we are given a file called `protected.zip`.

1. Analysis

The first thing I did was check what kind of file it was:

```

└─$ file protected.zip
protected.zip: Zip archive data  made by v6.3 UNIX, extract using at least v2.0, last modified Apr 02 2026 22:08:24, uncompressed size 36, method=store

```

So it is a zip file. When I tried to unzip it, it asked for a password, so it is password-protected. The hint "Jack the Ripper" made it clear that we would need **John the Ripper**, a well-known password cracking tool.

2. Decipher

To crack the password, I first needed to extract the hash from the zip file. For that I used **zip2john**, which is a tool that comes with John the Ripper and converts the zip into a format that John can work with. This also revealed a file named **flag.txt** inside the zip file.

```
└─$ zip2john protected.zip > hash.txt
ver 2.0 protected.zip/flag.txt PKZIP Encr: cmplen=48, decmplen=36, crc=06B9FFFC ts=B10C cs=06b9 type=0
```

Then I checked the content of **hash.txt**:

```
└─$ cat hash.txt
protected.zip/flag.txt:$pkzip$1*1*2*0*30*24*0b9fffc*0*26*0*30*06b9*9980c2639bf709209aea010e2af40a2e1e95e1106a4a43650f605fb7efc8af6f8324ff48951a21471bd4e62fca0154e*$/pkzip$:flag.txt:protected.zip::protected.zip
```

With the hash ready, I ran John the Ripper using the **rockyou.txt** wordlist.

```
└─$ john hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 12 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Lejoshg (protected.zip/flag.txt)
1g 0:00:00:00 DONE (2026-04-13 10:16) 1.136g/s 12427Kp/s 12427Kc/s 12427KC/s Lightyagami..LM741
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

John found the password: **Lejoshg**. I then unzipped the file using that password:

```
└─$ unzip -P Lejoshg protected.zip
Archive:  protected.zip
extracting:  flag.txt
```

```
└─$ cat flag.txt
fsuCTF{John_Work_Well_With_Rockyou}
```

Flag: **fsuCTF{John_Work_Well_With_Rockyou}**